TeachForge — Blueprint del proyecto

DSC HACK 2026 · Reto «Educación 2036» · Grieta 03: docentes sin relevo

Tesis: el criterio pedagógico de un docente no debería jubilarse con él.

0. En 30 segundos

TeachForge captura la experiencia real de docentes veteranos (qué error cometen los alumnos, por qué ocurre y qué lo destraba), la convierte en casos estructurados y la usa para que docentes en formación practiquen en simulaciones de aula con estudiantes simulados. Cada intento deja evidencia en un perfil de competencias.

MVP del hackatón (un ciclo completo y desplegado, una materia, 2–3 casos reales):

Captura → Caso validado por el veterano → Simulación (3 turnos) → Evaluación por rúbrica con evidencia → Perfil de competencias.

Criterio	Peso	Qué le muestras al jurado
Problemática	30%	Dato + fuente, y lo que aprendiste de un docente real
Innovación	30%	La lectura: lo que se jubila no es contenido sino criterio pedagógico; y el ciclo que lo convierte en entrenamiento
Técnico y factibilidad	20%	Demo desplegada, estable, con fallback y costo por sesión medido
Pitch y caso de negocio	20%	Quién lo adopta (institutos pedagógicos, facultades de Educación, UGEL, redes de colegios) y un piloto concreto

1. El problema

Hoy: según la slide del reto, el 22% de los docentes de secundaria tiene 55 años o más (OCDE Education at a Glance 2026 y GRADE vía Gestión, 19/10/2026; Minedu vía La República, 05/07/2026). Ojo: ese dato mide edad, no jubilaciones. Dilo así.

Mecanismo (de la slide): se jubila una generación, la que entra no alcanza y la que se queda no fue formada para enseñar con IA.

Tu lectura no obvia: lo que se pierde no es contenido (está en libros y en la IA). Se pierde el conocimiento pedagógico del contenido: qué error comete el alumno, por qué lo comete y qué explicación lo destraba. Se aprende en años de aula y casi nunca se documenta (busca el concepto de Shulman, «pedagogical content knowledge», para citarlo bien).

Por qué crece hacia 2036:

1. La cohorte de 55+ llega a la jubilación en ese horizonte.
2. Con IA el contenido es casi gratis; el criterio para enseñarlo es lo escaso.
3. Los docentes nuevos necesitan práctica con casos reales, no solo cursos completados.

Tabla de evidencia (llénala hoy; lo no verificado no se afirma)

Afirmación	Qué buscar	Fuente probable	Estado
22% de docentes de secundaria con 55+	Abrir la fuente original y anotar tabla y página	OCDE EaG 2026 / GRADE / Minedu	Pendiente de verificar
La que entra no alcanza	Ingresantes a formación docente, vacantes, plazas	Minedu, INEI, SUNEDU	Pendiente
No formada para enseñar con IA	% de docentes capacitados en IA	Minedu, UNESCO, encuestas	Pendiente
Conocimiento tácito sin documentar	Tu entrevista con un docente veterano	Tu carpeta de evidencias	Por conseguir

2. La idea

Qué es: una plataforma que convierte relatos de aula de docentes con experiencia en casos pedagógicos validados por ellos mismos, y los usa para entrenar a docentes nuevos con simulaciones y retroalimentación con evidencia.

Qué NO es: un chatbot profe, un detector de IA ni un banco de planes de clase. El diferencial es la fuente (casos reales de un veterano, validados por él) y el ciclo (captura → simulación → evidencia → siguiente práctica).

Toque 2036 (opcional, recomendado): un «momento IA» en la simulación: un estudiante llega con una respuesta que le dio una IA, con un error plausible. El docente debe notarlo y usarlo para enseñar. Une la grieta 03 con la 02.

Ejemplo de uso

1. María (veterana, matemática de secundaria) cuenta cómo enseña ecuaciones de primer grado a alumnos con dificultades en operaciones básicas.

2. La app estructura el caso: error frecuente (pasan términos cambiando mal el signo), por qué ocurre, qué hizo ella (balanza con objetos), cómo supo que entendieron. María revisa, edita y valida.
3. Carlos (docente en formación) abre el caso, ve la escena y debe enseñarlo a 3 estudiantes simulados durante 3 turnos.
4. Recibe puntajes 0–3 por competencia con citas textuales de lo que dijo, y ve cómo lo hizo María.
5. Su perfil muestra qué competencia trabajar a continuación.

3. Alcance del MVP

Sí se construye:

- Captura guiada (texto, con dictado opcional) y extracción a caso estructurado.
- Pantalla de revisión y validación por el veterano, con casilla de consentimiento.
- Biblioteca de casos (marcando cuáles son reales y cuáles de ejemplo).
- Simulación de 3 turnos con 3 estudiantes simulados.
- Evaluación con rúbrica de 4 competencias (+1 opcional) con evidencia citada.
- Perfil de competencias con evolución por intento.
- Registro de tokens y costo por sesión.

No se construye (hoja de ruta; dilo con honestidad en el pitch): grafo de conocimiento, recuperación vectorial entre casos, orquestación multiagente, recomendación adaptativa longitudinal, gemelo digital completo. Con 2–3 casos no hace falta recuperación: pasas el caso entero al modelo.

Hipótesis que sí puedes tocar en 2 días:

- H1: los veteranos tienen conocimiento valioso que se puede capturar en minutos. Métrica: el veterano califica de 1 a 5 qué tan fiel es el caso a lo que quiso decir, y cuántas ediciones hizo.
- H2 (solo indicativa): practicar con un caso real mejora un segundo intento. Métrica: puntajes del intento 1 contra el 2 de una misma persona.
- H3 (perfil frente a certificados): NO la afirmes; queda como siguiente paso.

4. Arquitectura y stack

Recomendado: Next.js (App Router, TypeScript) + Tailwind + Supabase (Postgres) + Vercel. Un solo despliegue, gratis para empezar y fácil de mostrar.

Plan B si el viernes en la noche vas atrasada: Streamlit (Python) + Supabase, desplegado en Streamlit Community Cloud. Mismos prompts y mismo esquema.

Flujo:

- Navegador → rutas /api/extract, /api/simulate, /api/evaluate (servidor) → API del LLM → Supabase.
- La clave del LLM y la service key de Supabase viven solo en variables de entorno del servidor.
- Todas las salidas del modelo se validan con Zod; si falla, un reintento con el error de validación; si vuelve a fallar, respuesta de respaldo.

Modelos (cualquier proveedor sirve; deja el proveedor aislado en lib/llm.ts): un modelo más capaz para extracción y evaluación (por ejemplo Claude Sonnet 5.5, claude-sonnet-5-5) y uno más barato y rápido para los estudiantes simulados (por ejemplo Claude Haiku 5.5, claude-haiku-5-5). Verifica crédito y límites antes de empezar.

Voz: dictado en el navegador con la Web Speech API (SpeechRecognition, lang es-PE, funciona en Chrome). Si sobra tiempo, subir audio y transcribir con Whisper. Si no, solo texto: no pongas la demo en riesgo por la voz.

Fallback de demo: variable DEMO_MODE=true que sirve respuestas guardadas en /fixtures si la API falla. Grábate además un video corto de respaldo.

Costo y sostenibilidad: guarda tokens de entrada y salida por intento y calcula el costo estimado. Presenta el costo por sesión medido; es evidencia directa para «que pueda sostenerse».

5. Modelo de datos (supabase/schema.sql)

```
create extension if not exists pgcrypto;

create table cases (
  id uuid primary key default gen_random_uuid(),
  created_at timestamptz default now(),
  author_alias text not null,
  years_experience int,
  subject text not null,
  level text not null,
  topic text not null,
  raw_story text not null,
  structured jsonb not null,
  validated_by_author boolean default false,
  is_seed boolean default false,
  consent_at timestamptz
```

```
);

create table attempts (
  id uuid primary key default gen_random_uuid(),
  created_at timestamptz default now(),
  case_id uuid references cases(id) on delete cascade,
  learner_alias text not null,
  transcript jsonb not null,
  tokens_in int,
  tokens_out int,
  cost_usd numeric
);

create table evaluations (
  id uuid primary key default gen_random_uuid(),
  created_at timestamptz default now(),
  attempt_id uuid references attempts(id) on delete cascade,
  scores jsonb not null,
  summary text,
  next_focus text,
  human_reviewed boolean default false
);
```

Privacidad: usa alias, sin nombres reales ni datos personales. Sin login para el MVP; menciónalo como limitación.

6. Esquema del caso estructurado (lib/schemas.ts con Zod)

Claves del JSON:

- titulo, materia, nivel, tema, concepto_clave
- error_frecuente: descripcion, ejemplo_alumno, por_que_ocurre
- estrategia_que_funciono: descripcion, pasos (lista), condiciones
- lo_que_no_funciono
- senal_de_comprension
- contexto_aula
- perfiles_estudiantes: lista de nombre, perfil, como_se_manifiesta
- campos_faltantes: lista de preguntas para el docente (lo que no dijo)

Preguntas guiadas de la captura (5 a 8 minutos):

1. ¿Qué tema y nivel enseñabas?
2. ¿Qué error de los alumnos se repetía?
3. ¿Cómo te diste cuenta de que no entendían?

4. ¿Qué hiciste que sí funcionó y en qué condiciones (grupo, tiempo, materiales)?
5. ¿Qué intentaste que no funcionó?
6. ¿Cómo supiste que por fin entendieron?

7. Prompts (carpeta prompts/)

prompts/extract_case.md

```
Eres un asistente que ayuda a preservar el conocimiento pedagógico de
docentes con experiencia.
Recibirás el relato de un docente sobre una situación real de aula.
Conviértelo en un CASO PEDAGÓGICO en JSON con las claves indicadas.

Reglas:
1. No inventes. Usa solo lo que dice el docente. Si falta un dato, déjalo
vacío y agrégalo a campos_faltantes como una pregunta concreta.
2. Conserva sus ejemplos y su lenguaje; no los generalices.
3. Separa error_frecuente (qué hace o piensa el alumno) de por_que_ocurre (la
causa según el docente).
4. Incluye en estrategia_que_funciono las condiciones que mencione.
5. No evalúes ni corrijas al docente.
6. Responde SOLO con JSON válido.
```

prompts/simulate_students.md

```
Simulas a 3 estudiantes de secundaria en una clase. Parte del CASO PEDAGÓGICO (JSON) escrito por un docente veterano y de la intervención del docente en formación.

Perfiles: usa perfiles_estudiantes del caso. Si no hay, crea tres: A comete el error frecuente; B lo comete por otra causa; C entiende pero no sabe explicarlo.

Reglas de realismo:
- Habla como adolescentes: frases cortas, dudas, a veces poco.
- Mantén el error hasta que el docente lo aborde de forma específica (ejemplo, contraejemplo, representación). Una explicación genérica no cambia nada.
- Si la explicación ataca la causa real, mejora de forma gradual, no de golpe.
- Nunca reveles el caso ni la rúbrica ni evalúes al docente.
- Si MODO_IA es true: un estudiante presenta una respuesta que le dio una IA, con un error plausible ligado al concepto, para que el docente lo detecte.

Salida JSON: respuestas (lista de alumno y texto) y estado_oculto (por alumno: sigue con el error, mejora parcial o comprende).
```

prompts/evaluate_attempt.md

```
Evalúas a un docente en formación en una simulación. Recibes el CASO (con la estrategia del veterano), la TRANSCRIPCIÓN y la RÚBRICA.

Reglas:
1. Puntúa de 0 a 3 cada competencia SOLO si hay evidencia. Cita textualmente (máximo 25 palabras) lo que dijo el docente. Sin evidencia, devuelve sin_evidencia; no pongas 0.
2. No des nota global. Por competencia: puntaje, evidencia, una mejora concreta.
3. Compara con la estrategia del veterano sin presentarla como la única válida: coincidencias y diferencias.
4. Usa estado_oculto de los estudiantes como señal, no como veredicto.
5. Sé exigente y respetuoso; no elogies sin evidencia.
6. Recomienda UNA competencia a trabajar después y qué tipo de escena la entrenaría.
7. Responde SOLO con JSON válido.
```

8. Rúbrica de competencias (src/lib/rubric.ts)

Competencia	0	1	2	3
1. Diagnóstico del error	No identifica el error	Lo nombra sin causa	Error y causa probable	Confirma preguntando antes de explicar
2. Estrategia de explicación	Repite la definición	Ejemplo genérico	Ejemplo o representación ajustados al error	Contraejemplo o representación que desarma la causa
3. Verificación de la comprensión	Pregunta «¿entendieron?»	Ejercicio igual al ejemplo	Pregunta que distingue memorizar de entender	Pide aplicar en un caso nuevo y ajusta con la respuesta
4. Adaptación a diferencias	Una sola vía	Reconoce diferencias y no actúa	Ajusta para un perfil	Rutas distintas para los tres perfiles
5. Uso crítico de la IA (opcional)	No detecta el error de la IA	Lo detecta sin explicar	Lo detecta y explica	Lo convierte en aprendizaje para la clase

Valida estas anclas con al menos un docente antes del sábado y anota sus cambios en la evidencia.

9. Estructura del repositorio

```
teachforge/
├─ README.md
├─ LICENSE                  (MIT)
├─ .env.example
├─ .gitignore
├─ docs/
|  ├─ problema-y-evidencia.md
|  ├─ arquitectura.md
|  ├─ rubrica.md
|  ├─ pitch.md
|  └─ declaracion-de-ia-y-trabajo-previo.md
├─ prompts/
|  ├─ extract_case.md
|  ├─ simulate_students.md
```

```
|  └─ evaluate_attempt.md
├─ supabase/
|  ├─ schema.sql
|  └─ seed.sql              (casos de ejemplo marcados is_seed = true)
├─ fixtures/                (respuestas guardadas para DEMO_MODE)
├─ src/
|  ├─ app/
|  |  ├─ page.tsx           (elige rol: veterano o en formación)
|  |  ├─ capture/page.tsx
|  |  ├─ cases/page.tsx
|  |  ├─ simulate/[caseId]/page.tsx
|  |  ├─ profile/page.tsx
|  |  └─ api/{extract,simulate,evaluate}/route.ts
|  ├─ lib/{llm.ts,db.ts,schemas.ts,rubric.ts,cost.ts}
|  └─ components/
└─ tests/                   (validación de esquemas y rúbrica)
```

Comandos para arrancar:

```
npx create-next-app@latest teachforge --typescript --tailwind --app
cd teachforge
npm i zod @supabase/supabase-js recharts
npm i @anthropic-ai/sdk   # o el SDK de tu proveedor
git init && git add . && git commit -m 'chore: init TeachForge'
gh repo create teachforge --public --source=. --push
```

.env.example:

```
LLM_API_KEY=
NEXT_PUBLIC_SUPABASE_URL=
NEXT_PUBLIC_SUPABASE_ANON_KEY=
SUPABASE_SERVICE_KEY=
DEMO_MODE=false
```

Buenas prácticas para el jurado del repo: commits pequeños y frecuentes (convencionales: feat, fix, docs), README claro, licencia, capturas de pantalla, y una sección que diga qué funciona hoy y qué es hoja de ruta. Si reutilizas código de proyectos anteriores (por ejemplo workforce-shift), decláralo en docs/declaracion-de-ia-y-trabajo-previo.md y consulta a los organizadores si está permitido.

10. Borrador del README

1. Título y tesis en una línea.
2. El problema con dato y fuente (la tabla de evidencia verificada).

3. Qué hace TeachForge y qué no es.
4. Demo desplegada (URL) y video de respaldo.
5. Cómo funciona (diagrama del ciclo) y stack.
6. Qué funciona hoy y qué es hoja de ruta.
7. Evidencias de uso (resumen anónimo) y aprendizajes.
8. Cómo correrlo en local.
9. Ética: consentimiento, alias, sin datos personales, el docente valida su propio caso.
10. Declaración de uso de IA y de trabajo previo.

11. Plan por bloques

Cuello de botella: conseguir al docente veterano y a un docente en formación. Escribe a tus grupos ya, en paralelo a todo lo demás.

Viernes:

- B0 (45 min): crear repo, desplegar un hola mundo en Vercel, crear el proyecto Supabase, probar la clave del LLM, pedir los docentes.
- B1 (2 h): captura guiada, extracción, pantalla de revisión y validación, guardado.
- B2 (2,5 h): simulación de 3 turnos y evaluación con evidencia citada.
- B3 (1,5 h): perfil de competencias, biblioteca de casos, estilos mínimos.
- B4: sesión de captura con el veterano (idealmente en este tramo) y prueba de uso con un docente en formación, con pantalla compartida.
- B5: validación Zod con reintento, DEMO_MODE, registro de costo, README, video de respaldo.
- Duerme algo antes del sábado.

Sábado:

- Hasta las 8:30: corregir lo que viste en las pruebas, ordenar la carpeta de evidencias.
- 9:00 llegada con DNI físico y laptop; 10:00 checkpoint técnico; 13:00 almuerzo; 15:00 checkpoint de producto.
- **Sube todo al formulario hacia las 16:00.** A las 17:00 cierra y no acepta nada.

Si a mitad del viernes sin avance en B2, recorta: una sola competencia en el perfil y evaluación sin el momento IA.

12. Carpeta de evidencias

Reglas del reto: hasta 3 registros cuentan, al menos 1 es prueba de uso del producto, todos con la plantilla oficial (revisa tu correo de postulación para el acceso) y nada se modifica después del cierre.

Registro	Tipo	Qué pasa	Qué mides
1	Prueba de uso	El docente veterano usa Captura con tu app y valida su caso	Fidelidad del caso (1 a 5), número de ediciones, minutos, una frase suya
2	Prueba de uso	Un docente en formación (o cualquier docente) resuelve la simulación y ve su evaluación	Puntajes del intento 1 frente al 2, dónde se trabó, qué cambiaste después
3	Validación del problema	Entrevista corta a un docente o directivo sobre el relevo y lo que se pierde al jubilarse	Qué aprendiste que no sabías; una cita; cómo cambió tu diseño

Para cada registro: perfil anonimizado, consentimiento por escrito (basta un mensaje), fecha, captura o grabación con permiso, y una línea de «qué cambié por esto». Los registros extra van al anexo y no se evalúan.

Texto de consentimiento (adáptalo): «Participo voluntariamente en una prueba de TeachForge para DSC HACK. Autorizo usar mis respuestas de forma anónima y, si lo indico, una captura o grabación de la sesión. Puedo retirarme cuando quiera.»

13. Pitch por jurado

Gancho: «Cada año se jubilan docentes que saben cosas que ningún libro dice. Hoy esas cosas se van con ellos.»

1. **Problema (Frank Gago):** el dato con su fuente, el mecanismo y lo que aprendiste del docente real, con su cita.
2. **Lectura 2036 (Andre Cruz):** lo que se pierde es criterio, no contenido; con IA el contenido es gratis y el criterio escaso; el ciclo convierte experiencia en entrenamiento.
3. **Demo y técnica (José Carlos Guerra):** el caso de la veterana, la simulación y la evaluación con citas, en vivo; luego una slide con qué funciona hoy, qué es hoja de ruta, el fallback y el costo por sesión.

4. **Adopción (Javier Blanco):** quién lo usa (un instituto pedagógico o facultad de Educación), un piloto concreto (por ejemplo 5 veteranos, 3 casos cada uno, 20 docentes en formación) y qué medirías; límites honestos.

Pregunta cuánto dura el pitch y ensaya con cronómetro.

14. Riesgos y mitigaciones

Riesgo	Mitigación
No consigues veterano a tiempo	Pídelo ya; casos de ejemplo marcados como tal, nunca presentados como reales
El modelo inventa pedagogía	Extracción sin inventar, validación del veterano y evaluación con citas textuales
Estudiantes simulados poco realistas	Anclarlos al error documentado y mostrar cuándo mejoran gradualmente
Falla la API en la demo	DEMO_MODE con fixtures y video de respaldo
Privacidad y crédito de los veteranos	Alias, consentimiento, validación y atribución como autor si lo desea
Exceso de alcance	Recortar a una competencia y un caso; el ciclo completo es lo que importa
Afirmar de más	H3 y mejora de calidad docente quedan como siguiente paso

15. Recursos y checklist de cuentas

- [ ] GitHub (repo público, licencia MIT, temas y descripción)
- [ ] Vercel conectado al repo
- [ ] Proyecto Supabase con schema.sql aplicado
- [ ] Clave de API del LLM con crédito verificado
- [ ] Plantilla de la carpeta de evidencias (correo de postulación)
- [ ] Herramienta para grabar la pantalla en las pruebas (la de Meet o Zoom basta)
- [ ] Documentación a mano: Next.js App Router, Supabase, Zod, Recharts, Web Speech API
- [ ] Fuentes a verificar: OCDE Education at a Glance 2026, GRADE, Minedu, INEI
- [ ] Concepto para citar: pedagogical content knowledge (Shulman)

16. Checklist final de entregables (antes de las 16:00 del sábado)

- [ ] URL del repo público
- [ ] URL de la presentación (pitch)
- [ ] URL de la demo desplegada y funcionando (probada desde otro dispositivo)
- [ ] Carpeta de evidencias con registros en la plantilla (al menos una prueba de uso)
- [ ] Formulario enviado
- [ ] DNI físico y laptop en la mochila